

PRACA DYPLOMOWA MAGISTERSKA

Architektura mikroservisów i rozwiązania chmurowe w ekosystemie .NET

*Microservices Architecture and Cloud Solutions in the
.NET Ecosystem*

Mariusz Trzeciak

Nr albumu: 133583

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

*Niniejszym chciałbym serdecznie podziękować
Promotorowi, dr inż. Bartoszowi Kowalczykowi,
za cenne wskazówki i cierpliwość
w trakcie realizacji niniejszej pracy.*

Spis treści

Wstęp	3
Cel pracy	3
Zakres pracy	3
1 Wprowadzenie do architektury mikroserwisów w ekosystemie .NET	5
1.1 Definicja i charakterystyka architektury mikroserwisowej	5
1.2 Porównanie z architekturą monolityczną	6
1.3 Zalety i wyzwania mikroserwisów	8
1.3.1 Zalety architektury mikroserwisowej	8
1.3.2 Wyzwania architektury mikroserwisowej	9
1.4 Ekosystem .NET w kontekście mikroserwisów	10
2 Mechanizmy komunikacji w architekturze mikroserwisowej	13
2.1 Komunikacja synchroniczna	13
2.1.1 REST API w ASP.NET Core	13
2.1.2 gRPC - architektura i implementacja	15
2.1.3 GraphQL jako alternatywa	16
2.2 Komunikacja asynchroniczna	17
2.2.1 Message Brokers – RabbitMQ i Azure Service Bus	17
2.2.2 Event-driven Architecture	19
2.3 Komunikacja w czasie rzeczywistym	20
2.3.1 WebSockets	20
2.3.2 Server-Sent Events	21
2.3.3 gRPC Streaming	22
2.4 Podsumowanie	23
3 Podsumowanie	25
Bibliografia	27
Dodatek. Zawartość dołączonej płyty	30
Spis rysunków	31

Spis tabel	32
Listing	33
Streszczenie	34
Summary	36
Słowa kluczowe	37

Wstęp

W ostatnim czasie obserwuje się intensywny rozwój [Proszę uzupełnić].

Niestety [Proszę wpisać wadę lub niedogodność istniejących rozwiązań].

W związku z tym, ciekawym wydaje się zaprojektowanie i zrealizowanie [Proszę uzupełnić].

Cel pracy

Celem niniejszej pracy jest zaprojektowanie i zrealizowanie [Proszę uzupełnić].

Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu [Proszę uzupełnić],
- ▶ porównanie [Proszę uzupełnić],
- ▶ opracowanie założeń projektowych dotyczących [Proszę uzupełnić],
- ▶ implementację [Proszę uzupełnić],
- ▶ przetestowanie [Proszę uzupełnić].

Rozdział 1

Wprowadzenie do architektury mikroserwisów w ekosystemie .NET

Niniejszy rozdział stanowi wprowadzenie do tematyki architektury mikroserwisów w kontekście nowoczesnego wytwarzania oprogramowania, ze szczególnym uwzględnieniem ekosystemu platformy .NET. Omówiona zostaje ewolucja podejść architektonicznych, od systemów monolitycznych po struktury usług rozproszonych, oraz kluczowe pojęcia i zasady projektowe, które stanowią fundament dalszych rozważań prowadzonych w kolejnych rozdziałach niniejszej pracy. Zaprezentowane definicje, zestawienia porównawcze i charakterystyki technologiczne służą jako punkt odniesienia dla badań eksperymentalnych opisanych w rozdziałach ?? i ?? oraz dla studium przypadku omówionego w rozdziale ??.

1.1 Definicja i charakterystyka architektury mikroserwisowej

Architektura mikroserwisów (ang. *microservices architecture*) definiowana jest jako podejście do projektowania systemu informatycznego w postaci zbioru małych, autonomicznych usług, z których każda realizuje ściśle określoną funkcję biznesową [3]. Zgodnie z definicją zaproponowaną przez Martina Fowlera i Jamesa Lewisa, mikroserwis stanowi niezależnie wdrażalną jednostkę oprogramowania uruchamianą we własnym procesie systemowym, komunikującą się z pozostałymi komponentami za pośrednictwem lekkich mechanizmów wymiany danych, najczęściej interfejsów HTTP/REST lub kolejek komunikatów [4]. Każda usługa organizowana jest wokół konkretnych zdolności biznesowych (ang. *business capabilities*), co oznacza, że jej granice logiczne wyznacza dziedzina problemu, a nie warstwa techniczna systemu. Architektura ta umożliwia niezależne skalowanie, wdrażanie i utrzymywanie poszczególnych komponentów, co bezpośrednio przekłada się na zwiększenie zwinności procesu twórczego oraz ograniczenie ryzyka kumulowania się zmian w jed-

nym, niepodzielnym artefakcie wdrożeniowym.

Newman podkreśla, że *niezależna wdrażalność* (ang. *independent deployability*) – możliwość wdrożenia pojedynczej usługi bez konieczności jednoczesnego wdrażania jakiegokolwiek innego komponentu systemu – jest nadrzędną właściwością definiującą mikroserwis [3]. Autor wskazuje, że osiągnięcie tej właściwości wymaga utrzymania luźnego sprzężenia między usługami, rozumianego jako minimalizacja zakresu wiedzy, którą jedna usługa musi posiadać na temat wewnętrznego działania innej. Najczęstszą przyczyną utraty niezależności wdrożeniowej są w praktyce współdzielone biblioteki zawierające logikę biznesową oraz bezpośredni dostęp do bazy danych innego mikroserwisu – obydwa przypadki tworzą ukryte sprzężenie, które wymusza synchronizację wdrożeń.

Naturalną granicą dla pojedynczego mikroserwisu jest koncepcja *Bounded Context* (ograniczonego kontekstu) wywodząca się z metodyki Domain-Driven Design (DDD) [6]. W ramach takiego kontekstu obowiązuje spójny model dziedzinowy, a wszelkie interakcje z zewnętrznymi komponentami odbywają się poprzez jawnie zdefiniowane interfejsy kontraktowe. Z zasadą granic kontekstowych ściśle powiązany jest wzorzec *database-per-service*, zgodnie z którym każdy mikroserwis posiada wyłączną własność swojego magazynu danych [5]. Rozdzielenie warstwy składowania danych eliminuje sprzężenie schematu bazy danych pomiędzy usługami i umożliwia dobór technologii przechowywania danych optymalnej dla konkretnego przypadku użycia: relacyjnych baz danych, dokumentowych, grafowych lub magazynów klucz-wartość. Podejście to pozostaje w opozycji do antywzorca współdzielonej bazy danych (ang. *shared database antipattern*), w którym wiele usług operuje na wspólnym schemacie, powodując silne sprzężenie strukturalne i uniemożliwiając niezależny rozwój poszczególnych komponentów systemu.

Evans podkreśla ponadto, że efektywna dekompozycja systemu na ograniczone konteksty wymaga wypracowania *wspólnego języka* (ang. *Ubiquitous Language*) – spójnego słownika pojęć stosowanego zarówno przez deweloperów, jak i ekspertów domenowych w obrębie danego kontekstu [6]. Stosowanie wspólnego języka eliminuje niejednoznaczności semantyczne – identyczny termin (np. „klient”) może posiadać różne znaczenie w kontekście sprzedaży i w kontekście obsługi reklamacji – oraz gwarantuje, że model dziedzinowy wiernie odzwierciedla rzeczywistość biznesową, a nie jedynie wewnętrzną strukturę kodu. Brak wspólnego języka prowadzi do konieczności stosowania złożonych mapowań kontekstowych (ang. *context mapping*), w tym kosztownych warstw translacyjnych.

1.2 Porównanie z architekturą monolityczną

Architektura monolityczna (ang. *monolithic architecture*) charakteryzuje się budową systemu jako pojedynczej, spójnej jednostki wdrożeniowej. W takim podejściu wszyst-

kie moduły funkcjonalne (prezentacja, logika biznesowa oraz dostęp do danych) umieszczone są w jednym artefakcie wdrożeniowym, korzystającym ze wspólnej bazy danych i uruchamianym w ramach jednego procesu systemowego [3]. Skalowanie poziome monolitu jest możliwe wyłącznie poprzez replikację całej aplikacji, niezależnie od tego, który z jej komponentów faktycznie wymaga dodatkowych zasobów obliczeniowych. Wraz ze wzrostem rozmiaru bazy kodu źródłowego czas kompilacji, wdrożenia i przeprowadzania testów rośnie proporcjonalnie, co w konsekwencji prowadzi do wydłużenia cykli wytwórczych i ograniczenia efektywności zespołu deweloperskiego. Wszelkie zmiany, nawet te o bardzo ograniczonym zasięgu, wymagają ponownego zbudowania i wdrożenia całości systemu, co istotnie zwiększa ryzyko operacyjne każdego wdrożenia produkcyjnego.

W odróżnieniu od architektury monolitycznej, podejście mikroserwisowe zakłada dekompozycję systemu na niezależne jednostki wdrożeniowe, co w sposób zasadniczy zmienia proces tworzenia oprogramowania oraz jego późniejszą eksploatację. Różnice pomiędzy oboma paradygmatami są wielowymiarowe i obejmują aspekty związane ze skalowalnością, izolacją awarii, złożonością techniczną oraz elastycznością doboru technologii. Szczegółowe zestawienie wybranych kryteriów oceny obu podejść architektonicznych zostało przedstawione w Tabeli 1.1.

Tabela 1.1: Porównanie architektury monolitycznej i mikroserwisowej.

Kryterium	Architektura monolityczna	Architektura mikroserwisowa
Skalowalność	Całościowa, horyzontalna oraz wertykalna	Selektywna, niezależna dla każdej usługi
Wdrożenie	Jeden artefakt, pełne wdrożenie całości	Niezależne wdrożenie każdej usługi
Izolacja awarii	Brak (awaria może objąć całość systemu)	Ograniczona do serwisu (wzorzec Circuit Breaker)
Złożoność	Prosta infrastruktura, rosnąca złożoność kodu	Prosta jednostka, złożona infrastruktura
Elastyczność technologiczna	Jednolity stos technologiczny	Polyglot persistence i programming
Testowanie	Testy integracyjne obejmują całość systemu	Łatwe testy jednostkowe, trudniejsze E2E
Czas do rynku (małe projekty)	Krótki (niski narzut konfiguracyjny)	Dłuższy (wysoki narzut konfiguracyjny)

Jak wynika z zestawienia przedstawionego w Tabeli 1.1, architektura mikroserwisowa oferuje wyraźne korzyści w zakresie skalowalności, izolacji awarii i elastyczności technologicznej, jednak wiąże się ze znacznie wyższą złożonością operacyjną i infrastrukturalną. Na szczególną uwagę zasługuje fakt, że w przypadku projektów o ograniczonej skali czas niezbędny do uruchomienia rozwiązania opartego na monolicie jest istotnie krótszy, co czyni architekturę monolityczną uzasadnionym wy-

borem dla systemów o przewidywalnym zakresie funkcjonalnym i niewielkim obciążeniu ruchowym. Decyzja o wyborze podejścia architektonicznego powinna zatem wynikać z analizy wymagań niefunkcjonalnych, prognozowanego wzrostu obciążenia systemu oraz dojrzałości organizacyjnej i technicznej zespołu wytwórczego.

1.3 Zalety i wyzwania mikroserwisów

Architektura mikroserwisów dostarcza wymiernych korzyści w kontekście skalowalności, szybkości wdrożeń i odporności systemów na awarie, lecz jednocześnie wprowadza nietrywialną złożoność operacyjną, której koszt zarządzania nie powinien być bagatelizowany przy podejmowaniu decyzji architektonicznej. Zrozumienie zarówno zalet, jak i wyzwań związanych z tym paradygmatem jest niezbędne do świadomego projektowania systemów oraz skutecznego stosowania wzorców i narzędzi opisanych w dalszych rozdziałach niniejszej pracy.

1.3.1 Zalety architektury mikroserwisowej

- ▶ **Niezależne skalowanie zasobów:** każdy mikroserwis może być skalowany w sposób autonomiczny, proporcjonalnie do bieżącego obciążenia, co eliminuje konieczność alokowania zasobów obliczeniowych dla nieobciążonych komponentów i pozwala na optymalizację kosztów infrastruktury chmurowej.
- ▶ **Izolacja awarii i wzorzec Circuit Breaker:** awaria pojedynczej usługi nie propaguje się w sposób kaskadowy na pozostałe komponenty systemu. Wzorzec Circuit Breaker, zaimplementowany w ekosystemie .NET m.in. za pomocą biblioteki Polly [22], stanowi podstawę oficjalnych mechanizmów odpornościowych (pakiet `Microsoft.Extensions.Http.Resilience`) i jest ściśle zintegrowany z `HttpClientFactory` w najnowszych wersjach platformy .NET.
- ▶ **Elastyczność technologiczna:** poszczególne usługi mogą być implementowane z wykorzystaniem różnych języków programowania (ang. *polyglot programming*) i różnych technologii persystencji (ang. *polyglot persistence*), co umożliwia dobór narzędzi optymalnych dla danego przypadku użycia, a nie narzuconych przez globalny stos technologiczny projektu.
- ▶ **Szybsze cykle wytwórcze i niezależny deployment:** zmiany w obrębie jednej usługi mogą być budowane, testowane i wdrażane niezależnie od pozostałych komponentów, co radykalnie skraca czas dostarczania nowych funkcjonalności i poprawek do środowiska produkcyjnego oraz istotnie zmniejsza ryzyko każdego wydania.

- ▶ **Małe, zrozumiałe bazy kodu sprzyjające utrzymaniu:** ograniczony zakres odpowiedzialności każdego mikroserwisu sprzyja utrzymaniu przejrzystości kodu źródłowego, upraszcza proces wdrażania nowych członków zespołu deweloperskiego oraz ułatwia refaktoryzację lub zastąpienie usługi nową implementacją bez ryzyka naruszenia pozostałych komponentów systemu.

Pomimo wymienionych zalet, wdrożenie architektury mikroserwisowej wiąże się z szeregiem istotnych wyzwań technicznych i organizacyjnych, które wymagają zastosowania odpowiednich wzorców projektowych, narzędzi oraz metodyk. Ignorowanie tych wyzwań na etapie planowania architektury prowadzi do powstawania systemów o wysokiej złożoności operacyjnej, trudnych do diagnozowania, skalowania i utrzymania w dłuższym horyzoncie czasowym.

1.3.2 Wyzwania architektury mikroserwisowej

- ▶ **Złożoność operacyjna i zarządzanie infrastrukturą kontenerową:** uruchomienie dziesiątek lub setek niezależnych usług wymaga zastosowania platformy orkiestracji kontenerów (np. Kubernetes), automatyzacji wdrożeń w ramach potoków CI/CD, zarządzania konfiguracją i sekretami, co znacznie podnosi wymagania względem kompetencji operacyjnych i infrastrukturalnych zespołu.
- ▶ **Zarządzanie danymi rozproszonymi i spójność ostateczna:** wzorzec *database-per-service* eliminuje możliwość stosowania transakcji ACID obejmujących wiele usług, zamiast tego stosuje się wzorzec Saga i akceptuje spójność ostateczną (ang. *eventual consistency*), co oznacza, że system dopuszcza chwilową niespójność danych pomiędzy usługami, gwarantując jednocześnie, że w przypadku braku nowych aktualizacji wszystkie repliki osiągną ostatecznie ten sam stan. Podejście to wymaga przemyślanego projektowania przepływów biznesowych i precyzyjnej obsługi stanów pośrednich.
- ▶ **Komunikacja międzyserwisowa: latencja, timeouty, retry, idempotentność:** synchroniczne wywołania sieciowe wprowadzają dodatkową latencję, a ich niezawodność jest ograniczona przez zawodność sieci, konieczne jest zatem stosowanie polityk ponownych prób (ang. *retry*), timeoutów i idempotentnych operacji, a w przypadku komunikacji asynchronicznej, zapewnienie gwarancji dostarczenia komunikatów przez brokera.
- ▶ **Observability: rozproszone śledzenie i centralne logowanie:** diagnozowanie problemów w systemach rozproszonych wymaga korelacji zdarzeń z wielu usług, standardem branżowym jest stosowanie OpenTelemetry [10]

do zbierania śladów (ang. *traces*), metryk i logów oraz narzędzi takich jak Jaeger (wykorzystany w części praktycznej niniejszej pracy), Zipkin czy Azure Monitor do ich wizualizacji i analizy.

- **Bezpieczeństwo: mTLS w service mesh, zarządzanie tożsamością serwisów:** każda ścieżka komunikacji pomiędzy usługami stanowi potencjalny wektor ataku, wzajemne uwierzytelnianie serwisów realizowane za pomocą mTLS w ramach service mesh (np. Istio [43] lub Linkerd [44]), zarządzanie tożsamością przy użyciu standardów OAuth 2.0/OIDC oraz rotacja sekretów są niezbędnymi elementami architektury bezpieczeństwa systemów mikroserwisowych.

Nygard, analizując wzorce stabilności systemów produkcyjnych, wskazuje że w nieobecności wzorca Circuit Breaker awaria jednej usługi może wywołać kaskadową degradację całego systemu poprzez mechanizm wyczerpywania pul wątków (ang. *thread pool exhaustion*): wołające usługi blokują kolejne wątki w oczekiwaniu na odpowiedź niedostępnego komponentu, aż do całkowitego wyczerpania zasobów i utraty responsywności przez usługi dotychczas działające poprawnie [38]. Autor podkreśla, że projektowanie odporności systemu (ang. *resilience engineering*) powinno być świadomą decyzją architektoniczną, a nie reakcją na incydenty produkcyjne.

Wymienione wyzwania nie stanowią argumentu przemawiającego przeciwko stosowaniu architektury mikroserwisowej, lecz wskazują na konieczność świadomego i systematycznego zarządzania złożonością systemu rozproszonego. Większość opisanych problemów posiada ugruntowane rozwiązania w postaci wzorców architektonicznych i narzędzi ekosystemowych szczegółowo omówionych w kolejnych rozdziałach niniejszej pracy [3][5].

1.4 Ekosystem .NET w kontekście mikroserwisów

Platforma .NET przeszła znaczącą ewolucję na przestrzeni ostatnich dwóch dekad, a jej trajektoria rozwojowa jest ściśle powiązana z rosnącymi wymaganiami dotyczącymi budowania systemów chmurowych i rozproszonych [7]. Pierwotna wersja środowiska (.NET Framework) przeznaczona była wyłącznie dla systemu operacyjnego Windows i oferowała rozbudowane, lecz monolityczne środowisko uruchomieniowe, głęboko zintegrowane z systemem operacyjnym. Ograniczenia związane z brakiem wsparcia dla platform Linux i macOS, a także nieprzenośny charakter środowiska uruchomieniowego, stanowiły istotne bariery w kontekście budowania przenośnych, konteneryzowanych aplikacji chmurowych. Przełomowym momentem w historii platformy było wydanie .NET Core 1.0 w 2016 roku, wieloplatformowego i otwartego środowiska uruchomieniowego (ang. *open-source runtime*), zoptymalizowanego pod

kątem wydajności i wdrożenia w kontenerach Docker [7]. Unifikacja ekosystemu nastąpiła wraz z wydaniem .NET 5 w 2020 roku, eliminując historyczny podział na .NET Framework i .NET Core. Kolejne wydania (.NET 6 LTS, .NET 7, .NET 8 LTS, .NET 9 oraz najnowsza wersja .NET 10 wydana w 2025 roku) konsekwentnie ugruntowały pozycję platformy jako pierwszorzędnego środowiska do wytwarzania natywnych aplikacji chmurowych (ang. *cloud-native applications*). Wydanie .NET 10 wprowadziło dalsze optymalizacje w obszarze narzutu protokołu HTTP/2 oraz rozszerzone wsparcie dla kompilacji statycznej (AOT), co ma bezpośredni wpływ na wydajność mikroserwisów opartych na gRPC, analizowanych w niniejszej pracy. Platforma zapewnia obecnie natywne wsparcie dla tworzenia obrazów Docker bezpośrednio z poziomu procesów budowania oraz zaawansowane integracje z usługami chmurowymi Microsoft Azure.

Fundamentem tworzenia mikroserwisów HTTP w ekosystemie .NET jest platforma ASP.NET Core, oferująca wysokowydajny serwer Kestrel, wbudowany kontener wstrzykiwania zależności (ang. *dependency injection container*), uproszczony interfejs minimalnych API (ang. *Minimal API*) oraz elastyczny potok przetwarzania żądań oparty na wzorcu oprogramowania pośredniczącego (ang. *middleware*) [7]. Wyniki testów wydajnościowych opublikowanych w ramach projektu TechEmpower Framework Benchmarks konsekwentnie plasują ASP.NET Core w gronie najszybszych frameworków webowych [9], co czyni go uzasadnionym wyborem dla usług wymagających wysokiej przepustowości i niskiej latencji. Ekosystem komunikacyjny platformy uzupełniają dedykowane pakiety: natywne wsparcie dla protokołu gRPC realizowane przez bibliotekę `Grpc.AspNetCore` [13], a także MassTransit [18] stanowiący warstwę abstrakcji nad brokerami komunikatów takimi jak RabbitMQ i Azure Service Bus, umożliwiający asynchroniczną wymianę zdarzeń pomiędzy usługami w oparciu o wzorzec Publish/Subscribe. Dostęp do danych realizowany jest najczęściej za pośrednictwem Entity Framework Core, obiektowo-relacyjnego mapera (ORM) obsługującego różnorodnych dostawców baz danych, w tym SQL Server, PostgreSQL, MySQL i SQLite.

Ekosystem narzędziowy firmy Microsoft obejmuje ponadto szereg rozwiązań dedykowanych orkiestracji i uproszczeniu procesu tworzenia systemów rozproszonych [7]. Rozszerzenie .NET Aspire stanowi warstwę orkiestracji dla lokalnego środowiska deweloperskiego, automatyzując wykrywanie usług (ang. *service discovery*) oraz konfigurację odporności połączeń. Dapr (ang. *Distributed Application Runtime*) dostarcza natomiast zestaw bloków konstrukcyjnych dla aplikacji rozproszonych w modelu sidecar. Szczególne znaczenie dla niniejszej pracy posiada referencyjny projekt eShop (dawniej eShopOnContainers) opracowany przez Microsoft jako wzorcowa implementacja architektury mikroserwisów [8]. Projekt ten demonstruje zastosowanie kluczowych wzorców architektonicznych: wzorca Saga realizowanego przez mechanizm Outbox oraz bramy API. Architektura projektu eShop

stanowi bezpośrednią podstawę studium przypadku omówionego w Rozdziale 8, w którym analizowane są implementacje wybranych wzorców komunikacyjnych i ich wpływ na wydajność oraz odporność systemu.

Analiza mechanizmów komunikacji międzyserwisowej, zarówno synchronicznej (REST, gRPC), jak i asynchronicznej (kolejki komunikatów, Event Bus), stanowi przedmiot rozważań w kolejnych rozdziałach niniejszej pracy.

Rozdział 2

Mechanizmy komunikacji w architekturze mikroservisowej

Jak przedstawiono w Rozdziale 1, architektura mikroservisowa opiera się na zasadzie izolacji funkcjonalnej poszczególnych usług. Każdy mikroservis jest odpowiedzialny za odrębny obszar biznesowy i działa jako niezależny proces. Jednakże aby system składający się z wielu mikroservisów mógł realizować złożone przypadki użycia, niezbędna jest efektywna komunikacja międzyservisowa. Wybór odpowiedniego mechanizmu komunikacji ma kluczowe znaczenie dla wydajności, skalowalności i niezawodności całej architektury [5].

W ekosystemie .NET dostępne są różnorodne podejścia do komunikacji, które można podzielić na trzy główne kategorie: komunikację synchroniczną (ang. *synchronous communication*), komunikację asynchroniczną (ang. *asynchronous communication*) oraz komunikację w czasie rzeczywistym (ang. *real-time communication*). Każde z tych podejść ma swoje zalety, wady oraz typowe zastosowania, które zostaną omówione w kolejnych sekcjach niniejszego rozdziału.

2.1 Komunikacja synchroniczna

Komunikacja synchroniczna charakteryzuje się tym, że wywołujący serwis oczekuje na odpowiedź od serwisu docelowego przed kontynuowaniem własnego przetwarzania. Model ten jest intuicyjny i przypomina tradycyjne wywołania funkcji, co ułatwia jego implementację i debugowanie. Jednakże wprowadza również ścisłe sprzężenie czasowe między serwisami, ponieważ niedostępność jednego z nich może bezpośrednio wpłynąć na działanie pozostałych [3].

2.1.1 REST API w ASP.NET Core

Representational State Transfer (REST) to styl architektoniczny zaproponowany przez Roya Fieldinga w 2000 roku [11], który definiuje zestaw ograniczeń dla pro-

jektowania rozproszonych systemów hipermedialnych. REST API wykorzystuje protokół HTTP i jego semantykę (metody GET, POST, PUT, DELETE) do manipulacji zasobami reprezentowanymi w formacie JSON lub XML. W architekturze mikroserwisowej REST stanowi najpopularniejszy mechanizm komunikacji synchronicznej ze względu na swoją prostotę, uniwersalność oraz szerokie wsparcie narzędziowe [12].

Fielding w swojej dysertacji zidentyfikował sześć ograniczeń architektonicznych definiujących styl REST: architekturę klient-serwer (ang. *client-server*), bezstanowość (ang. *statelessness*), możliwość buforowania odpowiedzi (ang. *cacheability*), jednolity interfejs (ang. *uniform interface*), warstwowość systemu (ang. *layered system*) oraz opcjonalne przesyłanie kodu na żądanie (ang. *code on demand*) [11]. Spośród tych ograniczeń kluczowe znaczenie dla interoperacyjności posiada jednolity interfejs, który konkretyzuje się przez cztery zasady: identyfikację zasobów przez URI, manipulację zasobami przez ich reprezentacje, samoopisowe komunikaty zawierające metadane dotyczące formatu i dozwolonych operacji, oraz hipermedia jako mechanizm sterowania stanem aplikacji (ang. *HATEOAS*). W praktycznych implementacjach API zasada HATEOAS jest często pomijana na rzecz prostszego modelu żądanie-odpowiedź, co według Fieldinga czyni takie systemy jedynie *podobnymi* do REST, nie zaś w pełni zgodnym z jego definicją.

W ekosystemie .NET implementacja REST API jest realizowana za pomocą platformy ASP.NET Core. W najnowszych wersjach platformy (w tym .NET 10), dostępny jest uproszczony model definicji punktów końcowych określany mianem *Minimal API*, który charakteryzuje się mniejszym narzutem konfiguracyjnym oraz wyższą wydajnością przy starcie usługi. Listing 2.1 przedstawia definicję endpointu REST implementującego operację pobierania produktu w tym modelu.

```
1 app.MapGet("/api/products/{id}", async (ProductDbContext db, string id)
    =>
2 {
3     var product = await db.Products
4         .AsNoTracking()
5         .FirstOrDefaultAsync(p => p.Id == id);
6
7     return product is null ? Results.NotFound() : Results.Ok(product);
8 });
```

Listing 2.1: Definicja endpointu REST z wykorzystaniem Minimal API w .NET 10.

Do głównych zalet REST API w kontekście architektury mikroserwisowej należą: bezstanowość (ang. *statelessness*), która ułatwia skalowanie horyzontalne, wykorzystanie standardowego protokołu HTTP, co eliminuje potrzebę specjalistycznych bibliotek, oraz możliwość stosowania cache'owania odpowiedzi na poziomie infrastruktury sieciowej. REST API charakteryzuje się również luźnym sprzężeniem kontraktowym, ponieważ struktura JSON pozwala na ewolucję interfejsu bez ko-

nieczności równoczesnej aktualizacji wszystkich klientów.

Wśród ograniczeń REST API można wymienić: stosunkowo duży rozmiar przesyłanych danych (JSON jest formatem tekstowym o większym narzucie niż formaty binarne), brak wbudowanej specyfikacji kontraktu (choć OpenAPI/Swagger częściowo to kompensuje) oraz problemy z wydajnością w scenariuszach wymagających wielu zapytań do różnych endpointów (ang. *N+1 problem*). Ponadto tradycyjne implementacje REST często wykorzystują HTTP/1.1, który implementuje model request-response z ograniczeniami dotyczącymi multipleksowania połączeń. Nowsze implementacje (np. ASP.NET Core w wersji 10) wspierają HTTP/2, co częściowo eliminuje to ograniczenie, jednakże REST nie oferuje natywnego wsparcia dla streamingu w stopniu porównywalnym z gRPC.

2.1.2 gRPC - architektura i implementacja

gRPC to nowoczesny framework komunikacji oparty na protokole HTTP/2 oraz formacie serializacji Protocol Buffers, opracowany przez Google i udostępniony jako projekt open-source w 2015 roku [13]. W przeciwieństwie do REST, gRPC wykorzystuje binarną serializację oraz umożliwia multipleksowanie wielu równoległych wywołań w ramach jednego połączenia TCP. Framework ten jest szczególnie popularny w komunikacji wewnętrznej między mikroserwisami (ang. *inter-service communication*), gdzie wysoką wydajność i niską latencję stawia się przed łatwością integracji z klientami zewnętrznymi.

Centralnym elementem gRPC jest definicja kontraktu za pomocą języka Protocol Buffers (protobuf), który pozwala na jednoznaczną specyfikację typów danych i operacji serwisowych [14]. Kontrakt zapisany w pliku `.proto` jest następnie kompilowany do kodu źródłowego w wybranym języku programowania, co gwarantuje silne typowanie i eliminuje rozbieżności między klientem a serwerem. Listing 2.2 prezentuje przykładową definicję serwisu w formacie protobuf.

```
1 syntax = "proto3";
2
3 package ProductService;
4
5 service ProductService {
6     rpc GetProduct (ProductRequest) returns (ProductResponse);
7     rpc StreamProducts (ProductRequest) returns (stream ProductResponse);
8 }
9
10 message ProductRequest {
11     string id = 1;
12 }
13
14 message ProductResponse {
15     string id = 1;
```

```
16 string name = 2;
17 double price = 3;
18 }
```

Listing 2.2: Definicja serwisu gRPC w formacie Protocol Buffers.

W ekosystemie .NET implementacja gRPC jest wspierana przez pakiet `Grpc.AspNetCore`, który integruje framework gRPC z platformą ASP.NET Core. Serwisy gRPC działają jako standardowe middleware w pipeline'ie ASP.NET Core, co pozwala na współdzielenie konfiguracji, mechanizmów autoryzacji oraz logowania z innymi komponentami aplikacji. Dzięki zastosowaniu HTTP/2 gRPC oferuje niższą latencję w typowych scenariuszach produkcyjnych z ruchem w sieci rozległej (WAN) – rzędu 10–50 ms – w porównaniu do REST API (50–200 ms), różnica ta wynika przede wszystkim z binarnej serializacji i multipleksowania HTTP/2 [26]. gRPC obsługuje również zaawansowane wzorce komunikacji, takie jak streaming dwukierunkowy (ang. *bidirectional streaming*) [13]. Ograniczenie dotyczące braku bezpośredniego wsparcia w przeglądarkach internetowych jest eliminowane przez protokół *gRPC-Web*, którego analiza stanowi przedmiot Rozdziału ??.

Główne zalety gRPC obejmują: wysoką wydajność dzięki binarnej serializacji protobuf (typowo 3–7 razy mniejszy rozmiar danych niż JSON, w zależności od struktury komunikatów [14]), wsparcie dla multipleksowania i flow control na poziomie protokołu HTTP/2 [26], silne typowanie zapewniane przez automatyczną generację kodu oraz wbudowaną obsługę streamingu danych. gRPC integruje się również z mechanizmami deadline propagation oraz distributed tracing (np. OpenTelemetry), co ułatwia monitorowanie i diagnozowanie problemów w rozproszonych systemach.

Do wad gRPC należy zaliczyć: ograniczoną czytelność komunikatów (format binarny nie jest human-readable), trudności z debugowaniem w porównaniu do REST (brak możliwości testowania przez zwykłą przeglądarkę), ograniczone wsparcie w starszych infrastrukturach sieciowych, które mogą nie obsługiwać HTTP/2, oraz bardziej złożony proces ewolucji kontraktów, ponieważ zmiany w plikach `.proto` wymagają rekompilacji kodu klienta. gRPC jest zatem optymalne dla komunikacji wewnętrznej między mikroserwisami, lecz REST pozostaje preferowany dla publicznych API dostępnych dla zewnętrznych konsumentów.

2.1.3 GraphQL jako alternatywa

GraphQL to język zapytań dla API oraz środowisko wykonawcze stworzone przez Facebook (obecnie Meta) w 2012 roku i opublikowane jako standard open-source w 2015 roku [32]. W przeciwieństwie do REST, gdzie struktura odpowiedzi jest z góry zdefiniowana przez serwer, GraphQL pozwala klientowi na precyzyjne określenie, które pola danych ma otrzymać w odpowiedzi. Mechanizm ten eliminuje pro-

blemy over-fetching (pobieranie zbędnych danych) oraz under-fetching (konieczność wykonywania wielu zapytań), które są typowe dla REST API [3].

W ekosystemie .NET implementacja serwerów GraphQL jest realizowana głównie za pomocą biblioteki Hot Chocolate, która oferuje pełne wsparcie dla specyfikacji GraphQL oraz integrację z ASP.NET Core [15]. GraphQL definiuje schema opisującą dostępne typy, zapytania (ang. *queries*) oraz mutacje (ang. *mutations*), co stanowi samodokumentujący się kontrakt między klientem a serwerem. Mechanizm resolverów pozwala na efektywne pobieranie danych z wielu źródeł, w tym z różnych mikroserwisów, co czyni GraphQL atrakcyjną opcją dla implementacji warstwy API Gateway (ang. *Backend for Frontend – BFF*).

Zaletami GraphQL są: elastyczność po stronie klienta, która umożliwia optymalizację zapytań pod kątem konkretnych potrzeb interfejsu użytkownika, redukcja liczby zapytań sieciowych poprzez możliwość pobierania zagnieżdżonych danych w jednym wywołaniu oraz silne typowanie i introspekcja schematu, co ułatwia integrację i automatyczne generowanie dokumentacji. GraphQL wspiera również mechanizm subskrypcji (ang. *subscriptions*) do komunikacji w czasie rzeczywistym.

Do wad GraphQL należą: zwiększona złożoność implementacji po stronie serwera, problemy z cache'owaniem odpowiedzi (brak możliwości wykorzystania standardowego HTTP cache), ryzyko tzw. query complexity attacks (złośliwe lub przypadkowo zaprojektowane zapytania mogą przeciążyć serwer) oraz potrzeba dodatkowej warstwy abstrakcji, która może maskować faktyczne zależności między mikroserwisami. Ze względu na te ograniczenia GraphQL jest najczęściej stosowany jako warstwa agregacji w API Gateway, a nie do bezpośredniej komunikacji między mikroserwisami.

2.2 Komunikacja asynchroniczna

Komunikacja asynchroniczna eliminuje konieczność oczekiwania na natychmiastową odpowiedź od serwisu docelowego. W tym modelu nadawca wysyła wiadomość i kontynuuje swoje przetwarzanie, natomiast odbiorca przetwarza wiadomość w dogodnym dla siebie czasie. Komunikacja asynchroniczna wprowadza luźne sprzężenie czasowe między serwisami, co zwiększa odporność systemu na awarie pojedynczych komponentów oraz ułatwia skalowanie. Jednakże wiąże się również ze zwiększoną złożonością implementacji oraz wyzwaniem związanym z zapewnieniem eventual consistency (ang. *spójność ostateczna*) [19].

2.2.1 Message Brokers – RabbitMQ i Azure Service Bus

Message Broker to oprogramowanie pośredniczące (ang. *middleware*), które przyjmuje wiadomości od nadawców i dostarcza je do odbiorców zgodnie z określonymi

regułami routingu. Message Brokery implementują wzorce komunikacyjne takie jak kolejki (ang. *queues*) oraz tematy (ang. *topics/exchanges*), co umożliwia implementację komunikacji typu point-to-point oraz publish-subscribe. Kluczową zaletą wykorzystania Message Broker jest trwałość wiadomości (ang. *message persistence*), która gwarantuje, że wiadomość nie zostanie utracona nawet w przypadku awarii jednego z mikroservisów.

W ekosystemie .NET najpopularniejszymi rozwiązaniami są RabbitMQ oraz Azure Service Bus. RabbitMQ to otwartoźródłowy Message Broker implementujący protokół AMQP 0.9.1, charakteryzujący się wysoką wydajnością, bogatym wsparciem dla wzorców routingu oraz możliwością wdrożenia w dowolnej infrastrukturze [16]. Azure Service Bus to w pełni zarządzana usługa chmurowa (ang. *Platform as a Service*), która oferuje dodatkowe funkcjonalności takie jak automatyczne wykrywanie duplikatów wiadomości, scheduled delivery oraz integrację z ekosystemem Azure [17].

Do komunikacji z Message Brokerami w aplikacjach .NET często wykorzystuje się bibliotekę MassTransit, która dostarcza wysokopoziomową abstrakcję nad różnymi implementacjami brokerów i ułatwia implementację wzorców takich jak Saga, Outbox Pattern oraz Retry Policies [18]. Listing 2.3 przedstawia przykładową konfigurację MassTransit z RabbitMQ.

```
1 services.AddMassTransit(x =>
2 {
3     x.AddConsumer<OrderCreatedConsumer>();
4
5     x.UsingRabbitMq((context, cfg) =>
6     {
7         cfg.Host("rabbitmq://localhost", h =>
8         {
9             h.Username("guest");
10            h.Password("guest");
11        });
12
13        cfg.ConfigureEndpoints(context);
14    });
15 });
```

Listing 2.3: Konfiguracja MassTransit z RabbitMQ w aplikacji .NET.

Zalety stosowania Message Brokerów obejmują: odporność na awarie dzięki trwałości wiadomości, możliwość load balancingu poprzez dystrybucję wiadomości między wiele instancji konsumenta, wsparcie dla mechanizmów retry oraz dead-letter queues w przypadku niepowodzeń przetwarzania, oraz możliwość tymczasowego buforowania wiadomości w sytuacjach, gdy konsument jest przeciążony. Message Brokery umożliwiają również implementację wzorca *Transactional Outbox*, który gwarantuje atomowość operacji zapisu do bazy danych i wysłania wiadomości,

co w połączeniu z orkiestracją typu *Saga* pozwala na zachowanie spójności ostatecznej w złożonych procesach biznesowych (np. składanie zamówienia), co zostało zaimplementowane w części praktycznej niniejszej pracy.

Do wad Message Brokerów należy zaliczyć: zwiększoną złożoność operacyjną (konieczność utrzymania dodatkowej infrastruktury), wyższą latencję w porównaniu do komunikacji synchronicznej (typowo 100–500 ms), trudności w diagnozowaniu problemów ze względu na asynchroniczną naturę komunikacji oraz potrzebę implementacji mechanizmów idempotentności po stronie konsumentów, aby zapewnić poprawność przetwarzania w przypadku ponowienia dostawy wiadomości.

2.2.2 Event-driven Architecture

Event-driven Architecture (EDA) to styl architektoniczny, w którym mikroserwisy komunikują się poprzez publikowanie i subskrybowanie zdarzeń biznesowych (ang. *domain events*). Zdarzenie reprezentuje fakt, że coś się wydarzyło w systemie, np. „zamówienie zostało złożone” lub „produkt został dodany do magazynu”. W przeciwieństwie do tradycyjnego podejścia opartego na komendach (ang. *command-driven*), gdzie nadawca jawnie określa, co odbiorca ma zrobić, w EDA nadawca jedynie informuje o zdarzeniu, a decyzję o reakcji podejmują zainteresowani odbiorcy [19].

Kluczową cechą EDA jest luźne sprzężenie między producentami i konsumentami zdarzeń. Producent zdarzenia nie ma wiedzy o tym, które serwisy subskrybują dane zdarzenie, co ułatwia rozbudowę systemu o nowe funkcjonalności bez modyfikacji istniejącego kodu. Mechanizm ten wspiera zasadę otwarte-zamknięte (ang. *Open-Closed Principle*) z inżynierii oprogramowania [30][29]. EDA jest naturalnym wyborem dla systemów implementujących wzorzec CQRS (ang. *Command Query Responsibility Segregation*) oraz Event Sourcing [36], które szczegółowo zostaną omówione w kolejnych rozdziałach niniejszej pracy.

W implementacji EDA w ekosystemie .NET stosuje się Message Brokery (takie jak RabbitMQ lub Azure Service Bus) jako infrastrukturę dostarczania zdarzeń. Zdarzenia są modelowane jako niezmiennicze obiekty DTO (ang. *Data Transfer Object*) zawierające informacje o tym, co się wydarzyło oraz niezbędny kontekst biznesowy. Istotne jest, aby zdarzenia były nazwane w czasie przeszłym (np. `OrderCreatedEvent`, `PaymentProcessedEvent`), co podkreśla ich charakter jako faktów, które już miały miejsce.

Zalety EDA obejmują: wysoki stopień luźnego sprzężenia między mikroserwisami, ułatwioną rozszerzalność systemu poprzez dodawanie nowych subskrybentów bez modyfikacji producentów, naturalną reprezentację procesów biznesowych jako sekwencji zdarzeń oraz możliwość implementacji zaawansowanych wzorców takich jak event replay (ponowne przetworzenie historycznych zdarzeń) czy tempo-

ral queries (zapytania o stan systemu w określonym momencie czasu).

Do wad EDA należą: zwiększona złożoność w zrozumieniu przepływu danych, ponieważ reakcje na zdarzenia są rozproszone w różnych serwisach, trudności w debugowaniu i trasowaniu żądań (ang. *distributed tracing*), potencjalne problemy z kolejnością przetwarzania zdarzeń w systemach rozproszonych oraz konieczność obsługi eventual consistency, co oznacza, że stan systemu może być tymczasowo niespójny między różnymi mikroserwisami. EDA wymaga również starannego projektowania granic kontekstów (ang. *bounded contexts*) oraz struktury zdarzeń, aby uniknąć niepożądanych zależności między serwisami.

2.3 Komunikacja w czasie rzeczywistym

Komunikacja w czasie rzeczywistym (ang. *real-time communication*) obejmuje mechanizmy umożliwiające natychmiastowe dostarczanie danych między klientem a serwerem z minimalnym opóźnieniem. W architekturze mikroserwisowej jest wykorzystywana przede wszystkim w interfejsach użytkownika wymagających aktualizacji na żywo, takich jak dashboards monitorujące, aplikacje czatu czy systemy notyfikacji. Komunikacja real-time wymaga utrzymywania długotrwałych połączeń, co wprowadza dodatkowe wymagania infrastrukturalne związane ze skalowaniem i zarządzaniem stanem sesji.

2.3.1 WebSockets

WebSocket to protokół komunikacyjny zdefiniowany w standardzie RFC 6455, który zapewnia pełnodupleksową (ang. *full-duplex*) komunikację dwukierunkową przez pojedyncze, długotrwałe połączenie TCP [20]. W przeciwieństwie do HTTP, gdzie klient zawsze inicjuje komunikację, WebSocket umożliwia serwerowi aktywne wysyłanie wiadomości do klienta bez wcześniejszego żądania. Protokół WebSocket rozpoczyna się od tzw. handshake HTTP, który następnie jest uaktualniany (ang. *upgrade*) do połączenia WebSocket, co zapewnia kompatybilność z istniejącą infrastrukturą webową.

W ekosystemie .NET implementacja WebSockets jest realizowana za pomocą biblioteki ASP.NET Core SignalR, która dostarcza wysokopoziomą abstrakcję nad protokołem WebSocket oraz automatyczne fallbacki do starszych technologii (Server-Sent Events, Long Polling) w przypadku, gdy WebSocket nie jest dostępny [21]. SignalR implementuje wzorzec Hub, który centralizuje logikę komunikacji real-time i umożliwia wywołania metod zarówno z serwera do klienta, jak i z klienta do serwera. Listing 2.4 przedstawia przykładowy Hub w SignalR.

```
1 public class NotificationHub : Hub
2 {
```

```

3     public async Task SendNotification(string user, string message)
4     {
5         await Clients.All.SendAsync("ReceiveNotification", user, message);
6     }
7
8     public override async Task OnConnectedAsync()
9     {
10        await Groups.AddToGroupAsync(Context.ConnectionId, "Users");
11        await base.OnConnectedAsync();
12    }
13 }

```

Listing 2.4: Implementacja SignalR Hub dla komunikacji real-time.

Zalety WebSockets obejmują: bardzo niską latencję (poniżej 10 ms) dzięki eliminacji narzutu związanego z nawiązywaniem nowego połączenia dla każdego żądania, efektywne wykorzystanie zasobów sieciowych poprzez ponowne użycie tego samego połączenia TCP, możliwość dwukierunkowej komunikacji inicjowanej zarówno przez klienta, jak i serwer, oraz wsparcie dla mechanizmów grupowania klientów (ang. *groups*), co ułatwia implementację komunikacji typu multicast.

Do wad WebSockets należy zaliczyć: trudności ze skalowaniem horyzontalnym ze względu na stanowość połączeń (wymagane są mechanizmy typu sticky sessions lub backplane), zwiększone zużycie zasobów serwerowych związane z utrzymywaniem tysięcy równoczesnych połączeń, problemy z load balancingiem w niektórych infrastrukturach sieciowych oraz brak wbudowanej obsługi reconnect logic w przypadku utraty połączenia (choć SignalR to kompensuje). WebSockets są optymalne dla aplikacji wymagających stałej wymiany danych w obu kierunkach, lecz mogą być nadmierne dla scenariuszy, gdzie aktualizacje są rzadkie.

2.3.2 Server-Sent Events

Server-Sent Events (SSE) to technologia umożliwiająca jednokierunkowy streaming danych z serwera do klienta przez standardowe połączenie HTTP. W przeciwieństwie do WebSocket, SSE nie wymaga uaktualnienia protokołu i działa w modelu request-response HTTP, gdzie połączenie pozostaje otwarte, a serwer przesyła kolejne zdarzenia jako fragmenty odpowiedzi w formacie `text/event-stream`. SSE jest wspierane natywnie przez wszystkie nowoczesne przeglądarki poprzez interfejs `EventSource` w JavaScript.

SSE jest prostszą alternatywą dla WebSocket w scenariuszach, gdzie wymagana jest jedynie komunikacja od serwera do klienta, takich jak przesyłanie notyfikacji, aktualizacji statusu czy strumieni danych z IoT. W ASP.NET Core implementacja SSE opiera się na mechanizmie `Response.WriteAsync` z ustawionym

nagłówkiem `Content-Type: text/event-stream` oraz wyłączonym buforowaniem odpowiedzi. SSE automatycznie obsługuje reconnect in przypadku utraty połączenia, co jest zaletą w porównaniu do niskopoziomowych implementacji WebSocket.

Zalety SSE obejmują: prostotę implementacji (bazuje na standardowym HTTP), automatyczny mechanizm reconnect z możliwością wznowienia streamu od ostatnio otrzymanego zdarzenia, wsparcie w standardowych narzędziach HTTP (debuggery, proxy) oraz brak konieczności specjalnej konfiguracji infrastruktury sieciowej. SSE działa dobrze z mechanizmami cache'owania i kompresji HTTP.

Do wad SSE należą: jednokierunkowość komunikacji (od serwera do klienta), ograniczenie liczby równoczesnych połączeń w niektórych przeglądarkach (typowo 6 połączeń na domenę w HTTP/1.1), brak wsparcia dla binarnych formatów danych (SSE przesyła dane tekstowe) oraz większy narzut w porównaniu do WebSocket ze względu na nagłówki HTTP. SSE są optymalne dla dashboardów, monitoringu oraz notyfikacji, gdzie serwer jest głównym źródłem aktualizacji.

2.3.3 gRPC Streaming

Jak wspomniano w sekcji 2.1.2, gRPC wspiera zaawansowane wzorce komunikacji oparte na streamingu. gRPC definiuje trzy typy streamingu: *server streaming* (serwer przesyła sekwencję wiadomości w odpowiedzi na jedno żądanie klienta), *client streaming* (klient przesyła sekwencję wiadomości, a serwer odpowiada pojedynczą wiadomością) oraz *bidirectional streaming* (obie strony przesyłają sekwencje wiadomości niezależnie od siebie). Wszystkie te wzorce są realizowane przez jedno połączenie HTTP/2, co zapewnia wysoką efektywność.

gRPC streaming łączy zalety komunikacji real-time (niską latencję, dwukierunkowość) z zaletami gRPC (silne typowanie, binarną serializację, multipleksowanie). Jest szczególnie przydatne w scenariuszach takich jak przesyłanie dużych zbiorów danych w fragmentach, implementacja long-running operations z raportowaniem postępu czy synchronizacja stanu między mikroservisami. W przeciwieństwie do WebSocket, gRPC streaming jest zintegrowane z mechanikami deadline propagation, retry policies oraz distributed tracing, co ułatwia utrzymanie niezawodności w systemach rozproszonych.

Zalety gRPC streaming obejmują: wysoką wydajność dzięki HTTP/2 i protobufovi, silne typowanie kontraktów, wbudowaną obsługę flow control i backpressure, możliwość multipleksowania wielu streamów w ramach jednego połączenia oraz integrację z ekosystemem gRPC (authentication, monitoring). gRPC streaming jest naturalnym wyborem dla komunikacji wewnętrznej między mikroservisami wymagającej przesyłania strumieni danych.

Wady gRPC streaming są analogiczne do wad samego gRPC: ograniczona

czytelność komunikatów, trudności z debugowaniem, potrzeba wsparcia HTTP/2 w infrastrukturze oraz złożoność implementacji w porównaniu do prostszych rozwiązań jak SSE. gRPC streaming jest optymalne w scenariuszach wewnętrznych, gdzie wymagana jest wysoka wydajność i niezawodność, lecz nie jest odpowiednie dla publicznych API dostępnych dla zewnętrznych klientów webowych.

2.4 Podsumowanie

W niniejszym rozdziale przedstawiono główne mechanizmy komunikacji stosowane w architekturze mikroserwisowej w ekosystemie .NET. Omówiono komunikację synchroniczną (REST API, gRPC, GraphQL), komunikację asynchroniczną (Message Brokers, Event-driven Architecture) oraz komunikację w czasie rzeczywistym (WebSockets, Server-Sent Events, gRPC Streaming). Każdy z tych mechanizmów ma swoje specyficzne zastosowania, zalety oraz ograniczenia, które należy uwzględnić podczas projektowania systemu.

Tabela 2.1 przedstawia syntetyczne porównanie omawianych mechanizmów pod względem latencji, stopnia sprzężenia, złożoności implementacji oraz typowych przypadków użycia.

Tabela 2.1: Porównanie mechanizmów komunikacji w architekturze mikroserwisowej (wartości latencji orientacyjne na podstawie [3]).

Mechanizm	Latencja	Sprzężenie	Złożoność	Typowe przypadki użycia
REST API	Średnia (50–200 ms)	Luźne	Niska	Publiczne API, operacje CRUD, integracje zewnętrzne
gRPC	Niska (10–50 ms)	Średnie	Średnia	Komunikacja wewnętrzna, wysokowydajne API, integracje wewnętrzne
Message Broker	Dostarczenie: <5 ms (LAN); łącznie z przetwarzaniem: 100–500 ms	Bardzo luźne	Wysoka	Asynchroniczne workflows, eventual consistency, procesy długotrwałe
WebSocket (SignalR)	Bardzo niska (<10 ms)	Ścisłe (stanowe)	Wysoka	Chat, notyfikacje real-time, dashboardy na żywo
SSE	Niska (10–50 ms)	Średnie	Średnia	Notyfikacje, monitoring, aktualizacje jednostronne
gRPC Streaming	Bardzo niska (<10 ms)	Średnie	Wysoka	Streaming danych, long-running operations, synchronizacja stanu

Wybór odpowiedniego mechanizmu komunikacji powinien być podyktowany

zarówno wymaganiami funkcjonalnymi (latencja, przepustowość, kierunkowość komunikacji), jak i niefunkcjonalnymi (skalowalność, niezawodność, łatwość utrzymania). W praktyce systemy mikroserwisowe często wykorzystują kombinację różnych mechanizmów: REST lub gRPC dla operacji synchronicznych wymagających natychmiastowej odpowiedzi, Message Brokery dla procesów asynchronicznych i długotrwałych, oraz WebSocket lub SSE dla interfejsów użytkownika wymagających aktualizacji w czasie rzeczywistym.

Szczegółowe zagadnienia związane z zapewnieniem niezawodności komunikacji, w tym implementacja wzorców Circuit Breaker, Retry oraz Saga, zostaną omówione w kolejnych rozdziałach niniejszej pracy. Ponadto w rozdziałach ?? oraz ?? przedstawione zostaną wyniki eksperymentów wydajnościowych porównujących REST API oraz gRPC.

Rozdział 3

Podsumowanie

W ramach niniejszej pracy:

- ▶ zebrano wiadomości z zakresu [Proszę wpisać tematykę],
- ▶ opisano istniejące rozwiązania z zakresu [Proszę wpisać],
- ▶ zaprojektowano autorski system informatyczny [Proszę wpisać pełną nazwę systemu],
- ▶ zaimplementowano autorski system informatyczny [Proszę wpisać pełną nazwę systemu],
- ▶ przetestowano [Proszę wpisać, co przetestowano w pracy].

Zaprojektowany i zrealizowany w pracy autorski [Proszę wpisać pełną nazwę systemu] charakteryzuje się tym, że:

- ▶ ... [Proszę wpisać zaletę systemu],
- ▶ ... [Proszę wpisać zaletę systemu],
- ▶ .. [Proszę wpisać zaletę systemu].

Zaprojektowany i zrealizowany w pracy autorski [Proszę wpisać pełną nazwę systemu] można wykorzystać m.in. do:

- ▶ ... [Proszę wpisać potencjalną możliwość zastosowania],
- ▶ ... [Proszę wpisać potencjalną możliwość zastosowania],
- ▶ ... [Proszę wpisać potencjalną możliwość zastosowania].

Zaprojektowany i zrealizowany w pracy autorski [Proszę wpisać pełną nazwę systemu] można w przyszłości rozbudować o następujące funkcje:

- ▶ ... [Proszę wpisać potencjalną możliwość rozbudowy],
- ▶ ... [Proszę wpisać potencjalną możliwość rozbudowy],
- ▶ ... [Proszę wpisać potencjalną możliwość rozbudowy].

Podsumowując można stwierdzić, że [Proszę wpisać pełną nazwę systemu], ze względu na [Proszę raz jeszcze wymienić podstawową zaletę rozwiązania], może być niezwykle użyteczny w [Proszę krótko wpisać obszar zastosowań].

Bibliografia

- [1] Zasady pisania prac dyplomowych, <https://wiisi.pcz.pl/student-wiisi/vademecum-studenta/praca-dyplomowa>, stan na dzień: 26.10.2024
- [2] Strona internetowa Wydziału Informatyki i Sztucznej Inteligencji, <https://wiisi.pcz.pl>, stan na dzień: 26.10.2024
- [3] Newman S., *Building Microservices. Designing Fine-Grained Systems*, wyd. 2, O'Reilly Media, 2021.
- [4] Fowler M., Lewis J., Microservices — a definition of this new architectural term, <https://martinfowler.com/articles/microservices.html>, stan na dzień: 20.05.2025
- [5] Richardson C., *Microservices Patterns. With Examples in Java*, Manning Publications, 2018.
- [6] Evans E., *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, 2003.
- [7] Microsoft Corporation, .NET documentation, <https://learn.microsoft.com/pl-pl/dotnet/>, stan na dzień: 20.05.2025
- [8] Microsoft Corporation, eShop — a .NET reference application implementing an e-commerce site, <https://github.com/dotnet/eShop>, stan na dzień: 20.05.2025
- [9] TechEmpower, *TechEmpower Framework Benchmarks*, <https://www.techempower.com/benchmarks/>, stan na dzień: 20.05.2025
- [10] OpenTelemetry Authors, OpenTelemetry Documentation, <https://opentelemetry.io/docs/>, stan na dzień: 20.05.2025
- [11] Fielding R., *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine, 2000.
- [12] Fowler M., Richardson Maturity Model, <https://martinfowler.com/articles/richardsonMaturityModel.html>, stan na dzień: 20.05.2025
- [13] gRPC Authors, gRPC Documentation, <https://grpc.io/docs/>, stan na dzień: 20.05.2025

- [14] Google Developers, Protocol Buffers Documentation, <https://protobuf.dev/>, stan na dzień: 20.05.2025
- [15] ChilliCream, Hot Chocolate GraphQL Server for .NET, <https://chillicream.com/docs/hotchocolate>, stan na dzień: 20.05.2025
- [16] RabbitMQ, RabbitMQ Documentation, <https://www.rabbitmq.com/documentation.html>, stan na dzień: 20.05.2025
- [17] Microsoft Corporation, Azure Service Bus Documentation, <https://learn.microsoft.com/azure/service-bus-messaging/>, stan na dzień: 20.05.2025
- [18] MassTransit Project, MassTransit Documentation, <https://masstransit.io/>, stan na dzień: 20.05.2025
- [19] Fowler M., What do you mean by "Event-Driven"?, <https://martinfowler.com/articles/201701-event-driven.html>, stan na dzień: 20.05.2025
- [20] Fette I., Melnikov A., The WebSocket Protocol, RFC 6455, <https://www.rfc-editor.org/rfc/rfc6455>, stan na dzień: 20.05.2025
- [21] Microsoft Corporation, ASP.NET Core SignalR Documentation, <https://learn.microsoft.com/aspnet/core/signalr/>, stan na dzień: 20.05.2025
- [22] App-vNext, Polly – resilience and transient-fault-handling library, <https://github.com/App-vNext/Polly>, stan na dzień: 20.05.2025
- [23] gRPC Authors, gRPC-Web – gRPC for Web Clients, <https://grpc.io/docs/platforms/web/>, stan na dzień: 20.05.2025
- [24] gRPC Authors, grpc-web – gRPC for Web Clients (GitHub), <https://github.com/grpc/grpc-web>, stan na dzień: 20.05.2025
- [25] Envoy Proxy Authors, Envoy Proxy Documentation, <https://www.envoyproxy.io/docs/>, stan na dzień: 20.05.2025
- [26] Thomson M., Benfield C., HTTP/2, RFC 9113, <https://www.rfc-editor.org/rfc/rfc9113>, stan na dzień: 20.05.2025
- [27] Google Developers, Protocol Buffer Compiler – Code Generation Reference, <https://protobuf.dev/reference/>, stan na dzień: 20.05.2025
- [28] Microsoft Corporation, ASP.NET Core Blazor WebAssembly, <https://learn.microsoft.com/aspnet/core/blazor/>, stan na dzień: 20.05.2025

- [29] Martin R.C., *Agile Software Development, Principles, Patterns, and Practices*, Prentice Hall, 2002.
- [30] Meyer B., *Object-Oriented Software Construction*, wyd. 2, Prentice Hall, 1997.
- [31] Wiggins A., *The Twelve-Factor App*, <https://12factor.net/>, stan na dzień: 20.05.2025
- [32] GraphQL Foundation, *GraphQL Specification*, <https://spec.graphql.org/>, stan na dzień: 20.05.2025
- [33] Young G., *CQRS Documents*, https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf, stan na dzień: 20.05.2025
- [34] Fowler M., *CQRS*, <https://martinfowler.com/bliki/CQRS.html>, stan na dzień: 20.05.2025
- [35] Bogard J., *MediatR – Simple, unambitious mediator implementation in .NET*, <https://github.com/jbogard/MediatR>, stan na dzień: 20.05.2025
- [36] Event Store Ltd., *EventStoreDB Documentation*, <https://developers.eventstore.com/server/v24.2/>, stan na dzień: 20.05.2025
- [37] ThreeMammals, *Ocelot API Gateway Documentation*, <https://ocelot.readthedocs.io/>, stan na dzień: 20.05.2025
- [38] Nygard M.T., *Release It! Design and Deploy Production-Ready Software*, wyd. 2, Pragmatic Bookshelf, 2018.
- [39] Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>, stan na dzień: 20.05.2025
- [40] Burns B., Grant B., Oppenheimer D., Brewer E., Wilkes J., *Kubernetes: Up and Running*, wyd. 3, O'Reilly Media, 2022.
- [41] Microsoft Corporation, *Azure Kubernetes Service (AKS) Documentation*, <https://learn.microsoft.com/azure/aks/>, stan na dzień: 20.05.2025
- [42] Helm Authors, *Helm Documentation*, <https://helm.sh/docs/>, stan na dzień: 20.05.2025
- [43] Istio Authors, *Istio Documentation*, <https://istio.io/latest/docs/>, stan na dzień: 20.05.2025
- [44] Buoyant Inc., *Linkerd Documentation*, <https://linkerd.io/docs/>, stan na dzień: 20.05.2025
- [45] OpenTelemetry Authors, *OpenTelemetry .NET Documentation*, <https://opentelemetry.io/docs/languages/net/>, stan na dzień: 20.05.2025

Dodatek. Zawartość dołączonej płyty

Do niniejszej pracy dołączono płytę z następującą zawartością:

- ▶ Dokument pracy w formatach tex i pdf.
- ▶ Kod źródłowy zaprojektowanego i zrealizowanego w ramach pracy systemu.

Spis rysunków

Spis tabel

1.1	Porównanie architektury monolitycznej i mikroserwisowej.	7
2.1	Porównanie mechanizmów komunikacji w architekturze mikroserwisowej (wartości latencji orientacyjne na podstawie [3]).	23

Spis listingów

2.1	Definicja endpointu REST z wykorzystaniem Minimal API w .NET 10.	14
2.2	Definicja serwisu gRPC w formacie Protocol Buffers.	15
2.3	Konfiguracja MassTransit z RabbitMQ w aplikacji .NET.	18
2.4	Implementacja SignalR Hub dla komunikacji real-time.	20

Streszczenie

Praca pt. „*Architektura mikroserwisów i rozwiązania chmurowe w ekosystemie .NET*” poświęcona jest projektowaniu, implementacji oraz empirycznej analizie wydajności systemu rozproszonego zbudowanego w oparciu o platformę .NET 10. W ostatnim czasie obserwuje się intensywny rozwój protokołu gRPC jako alternatywy dla powszechnie stosowanego REST w komunikacji między mikroserwisami. Jednocześnie środowisko przeglądarek internetowych wymaga stosowania wariantu *gRPC-Web*, którego wydajność w komunikacji klient–backend pozostaje słabo zbadana empirycznie. W rozważanej pracy podjęto zatem próbę wypełnienia tej luki badawczej. W szczególności zebrano wiadomości z zakresu architektury mikroserwisów, mechanizmów komunikacji synchronicznej i asynchronicznej oraz wzorców projektowych w ekosystemie .NET, porównano trzy warianty komunikacji klient–backend: REST (HTTP/JSON), gRPC-Web przez proxy Envoy oraz gRPC-Web Direct bez pośrednika, opracowano środowisko testowe oparte na narzędziu k6 i platformie Docker Compose, zrealizowano aplikację referencyjną eShop złożoną z dwóch mikroserwisów (*ProductService* i *OrderService*) w technologii ASP.NET Core 10, przetestowano wydajność poszczególnych protokołów przy różnych poziomach obciążenia (10, 50 i 100 wirtualnych użytkowników) i rozmiarach ładunku (10–200 rekordów). Zaprojektowany i zaimplementowany system charakteryzuje się następującymi cechami: podejściem *contract-first* opartym na *Protocol Buffers* zapewniającym silną typizację w całym stosie technologicznym, kompleksową obserwowalnością obejmującą metryki (Prometheus, Grafana) i ślady rozproszone (Jaeger), implementacją wzorców klasy enterprise (Transactional Outbox, Saga State Machine). Dzięki temu zaprojektowany i zaimplementowany system może być wykorzystany w praktyce np. do: porównywania wydajności protokołów komunikacyjnych w architekturze mikroserwisowej .NET, demonstracji wzorców projektowych stosowanych w systemach rozproszonych, a także jako platforma testowa dla nowych scenariuszy komunikacji sieciowej.

Do realizacji praktycznej części pracy wykorzystano najnowsze narzędzia i technologie informatyczne, w tym m.in. *.NET 10* i *ASP.NET Core Minimal API*, framework gRPC z serializacją *Protocol Buffers*, framework frontendowy *Angular 20* z biblioteką *Connect RPC*.

Zaprojektowany i zaimplementowany system można w przyszłości przykła-

dowo rozbudować o następujące funkcje: wdrożenie w środowisku Kubernetes (Azure AKS) z automatycznym skalowaniem horyzontalnym, dodanie warstwy uwierzytelniania JWT i autoryzacji na poziomie serwisów oraz rozszerzenie macierzy testowej o scenariusze mutacji danych i testów odporności (chaos engineering).

Summary

This thesis, entitled “*Microservices Architecture and Cloud Solutions in the .NET Ecosystem*”, is devoted to the design, implementation, and performance analysis of a distributed system built on the .NET 10 platform. With the growing adoption of the gRPC protocol in backend communication, its browser-compatible variant *gRPC-Web* has emerged as a significant yet underexplored alternative to the ubiquitous REST protocol for client-backend communication. This work addresses that research gap. In particular, knowledge was gathered in the areas of microservices architecture, synchronous and asynchronous communication mechanisms, and design patterns used in the .NET ecosystem; three client-backend communication variants were compared: REST (HTTP/JSON), gRPC-Web via Envoy proxy, and gRPC-Web Direct without a proxy; a testing environment was built using the k6 load testing framework on Docker Compose; a reference eShop application consisting of two microservices (*ProductService* and *OrderService*) was implemented in ASP.NET Core 10; and the performance of each protocol was measured at different load levels (10, 50, and 100 virtual users) and payload sizes (10–200 records). The designed and implemented system is characterised by: a contract-first approach based on *Protocol Buffers* ensuring strong typing across the entire technology stack, comprehensive observability including metrics (Prometheus, Grafana) and distributed tracing (Jaeger), and implementation of enterprise patterns (Transactional Outbox, Saga State Machine). The system can be used in practice for comparing the performance of communication protocols in a .NET microservices architecture, demonstrating design patterns in distributed systems, and serving as a testing platform for new network communication scenarios. Future extensions may include deployment to a Kubernetes environment (Azure AKS) with horizontal autoscaling, addition of JWT authentication and service-level authorisation, and expansion of the test matrix to cover mutation scenarios and resilience testing.

Słowa kluczowe

mikroserwisy;
gRPC-Web;
REST API;
Protocol Buffers;
ASP.NET Core;
Envoy Proxy;
testy wydajnościowe;
architektura rozproszona;
.NET 10

Imię i nazwisko: Mariusz Trzeciak

Nr albumu: 133583

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Architektura mikroserwisów i rozwiązania chmurowe w ekosystemie .NET*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.